DAVID NDIGA DCS-02-861012024

ASSIGNMENT TWO

1.

```
#include<iostream>
#include<string>

using namespace std;

// class
class Animal {
Private:
  // Encapsulation, data is hidden using private only accessible by
     Public methods
  string name;
  int age;

Public:
  // constructor
Animal (string n, int a): name(n), age(a) {}
// Public methods to acess private data (Encapsulation)
string getName() const {
      return name;
  }
  int getAge() const {
     return age;
  }
// Abstraction - Hiding complex details and showing only essential details
virtual void ~~make~~ sound() const {
      cout << "makes sound" << endl;
  }
};
```

```
// Inheritance Dog inherits from Animal
class Dog : public Animal {
 public:
    string breed
 Dog (string n, int a, string b) : Animal (n,a), breed(b) {}
    string getBreed() const {
        return breed
    }
    // Polymorphism : Overriding speak method
    void Sound() {
      cout << "barks" << endl;
    }
};
int main () {
  // objects, creating instances of class
  Animal thisAnimal ("German shepherd", 5);
   Dog thisDog ("Puppy", 3);
  // function overloading
  thisAnimal.Sound();
  thisDog.Sound();

  return 0;
}
```

2. Virtual keywod

```
# include <iostream>
using namespace std;

class Shape {
 Public:
  // Declaring virtual allows derived classes to overide it through polymo
   virtual void calculateArea () {
        cout << "calculating area for shap." << endl;
   }
 ~class
};
class Circle : public Shap {
 Public:
  void calculateArea () override {
   // override is optional it override calculatArea
    cout << "calculating area for a circle." << endl;
   }
};
int main () {
 Shape myshape;
 Circle mycircle;

 myshape.calculateArea ();
 mycircle.calculateArea ();

   return 0;
}
```

operator Overloading

3.
```
#include <iostream>
using namespace std;
Class Vector {
Public:
  int x, y;
  vector (int x=0, int y=0) : x(x), y(y) {}
   vector operator +(const vector) const {
      return vector (this->x + other.x, this->y + other.y);
   }
   void display () const {
      cout << "vector(" << x << ", " << y << ")" << endl;
   }
};
int main () {
  vector v1(10, 20);
  vector V2(5, 7);
  return 0;
}
```

Function Overloading

```
#include <iostream>
using namespace std;
Class Print {
Public:
  void show (int i) {
    cout << "Integer:" << i << endl;
}
  void show (double d) {
     cout << "Double:" << d << endl;
}
  void show (string s) {
     cout << "String:" << s << endl;
}
};
```

```
int main(){
  Print obj;
   obj.show(10);
   obj.show(3.14);
   obj.show("Hello");
   return 0;
}
```

Function Overriding

```
#include<iostream>
using namespace std;
~~class Parent~~
class Car {
Public:
   void drive(){
    cout<<"Driving on Highway"<<endl;
   }
};
class Toyota : Public Car;
Public:
   void drive ()
   cout<<" A Toyota on the Highway"<<endl;
   }
};
int main (){
  Car Xcar;
 Toyota Xtoyota;

Xcar.drive();
Xtoyota.drive();
 return 0;
}
```

4. Multidimensional Array

```
# Include <iostream>
using namespace std;
int main () {
int grid[3] [3] = {
  {1, 2, 3 },
  {4 ; 5, 6 },
  { 7, 8, 9 }
};
cout << Demonstrating Multidimensional Array << endl;

for (int i=0; i<3; i++) {
    for (int j = 0; j<3; j++) {
      cout << grid [i] [j] << " ";
    }
    cout << endl;
  }
return 0;
}
```